

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя/Лабораторная работа

Выполнил:

Кащеев Максим Николаевич

Группа

JS211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Требования к приложению:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API (желательно посредством axios)
- Разумное деление на компоненты (продемонстрировать понимание компонентного подхода)
- Использование composable для выделения повторяющегося функционала в отдельные файлы

Ход работы

Структура проекта

Приложение мигрировано на Vue 3 с использованием Vite в качестве сборщика. Проект разделён на frontend (Vue SPA) и backend (Express + JSON-файл базы данных). Структура исходных файлов из ЛР1/ЛР2 включала отдельные HTML-страницы index.html, models.html, datasets.html, spaces.html, account.html, login.html, registration.html и JavaScript-модуль api.js с запросами к серверу на порту 3001. В Vue-версии каждая страница стала отдельным view-компонентом, навигация между ними организована через vue-router.

1. Подключение роутера

Роутер подключён с использованием библиотеки vue-router v4. Маршруты описаны в файле src/router/index.js. К приложению роутер подключается в src/main.js через вызов app.use(router).

Листинг src/router/index.js:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import ModelsView from '../views/ModelsView.vue'
import ModelDetailView from '../views/ModelDetailView.vue'
import DatasetsView from '../views/DatasetsView.vue'
```

```

import SpacesView from '../views/SpacesView.vue'
import LoginView from '../views/LoginView.vue'
import RegisterView from '../views/RegisterView.vue'
import AccountView from '../views/AccountView.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: HomeView },
    { path: '/models', component: ModelsView },
    { path: '/models/:id', component: ModelDetailView },
    { path: '/datasets', component: DatasetsView },
    { path: '/spaces', component: SpacesView },
    { path: '/login', component: LoginView, meta: { guestOnly: true } },
    { path: '/register', component: RegisterView, meta: { guestOnly: true } },
    { path: '/account', component: AccountView, meta: { requiresAuth: true } },
  ]
})

router.beforeEach((to, _, next) => {
  const token = localStorage.getItem('token')
  if (to.meta.requiresAuth && !token) return next('/login')
  if (to.meta.guestOnly && token) return next('/account')
  next()
})

export default router

```

Листинг src/main.js:

```

import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'

createApp(App)
  .use(createPinia())
  .use(router)
  .mount('#app')

```

В шаблоне корневого компонента App.vue размещён тег `<router-view />`, в котором отображается текущий view-компонент согласно активному маршруту:

```

<template>
  <AppNavbar />
  <router-view />

```

</template>

2. Работа с внешним API

Для взаимодействия с backend-сервером (Express, порт 3001) создан модуль `src/api/axios.js` с базовым инстансом `axios`. Отдельные модули (`models.js`, `datasets.js`, `spaces.js`) содержат функции для конкретных эндпоинтов.

Листинг `src/api/axios.js`:

```
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:3001',
  headers: { 'Content-Type': 'application/json' },
})

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

export default api
```

Листинг `src/api/models.js`:

```
import api from './axios'

export const getModels = (params = {}) => api.get('/api/models', { params })
export const getModelById = (id) => api.get(`/api/model/${id}`)
```

Аналогичные модули `datasets.js` и `spaces.js` реализованы по той же схеме для маршрутов `/api/datasets` и `/api/spaces`.

Backend-сервер принимает запросы по следующим маршрутам, реализованным в `Server.js` (JP2):^[1]

Метод	Маршрут	Описание
GET	<code>/api/models</code>	Список моделей, поддерживает <code>?search=</code> и <code>?category=</code>
GET	<code>/api/model/:id</code>	Модель по <code>id</code>

GET	/api/datasets	Список датасетов
GET	/api/spaces	Список spaces
POST	/auth/login	Авторизация
POST	/auth/register	Регистрация

3. Деление на компоненты

Приложение разделено на переиспользуемые компоненты и страницы-views. Страницы не содержат дублирующуюся логику - она вынесена в composables и дочерние компоненты.

Листинг переиспользуемого компонента `src/components/ItemCard.vue`:

```
<script setup>
defineProps({
  item: { type: Object, required: true },
  type: { type: String, default: 'model' }, // 'model' | 'dataset' | 'space'
})
const emit = defineEmits(['like'])
</script>

<template>
<div class="card">
  <span class="badge">{{ item.tagLabel }}</span>
  <h3>{{ item.title }}</h3>
  <p>{{ item.description }}</p>
  <div class="meta">
    <span>↓ {{ item.downloads }}</span>
    <button @click="emit('like', item.id)">♥ {{ item.likes }}</button>
  </div>
</div>
</template>
```

Компонент `ItemCard.vue` используется на страницах `ModelsView.vue`, `DatasetsView.vue` и `SpacesView.vue` через `v-for`, что исключает дублирование разметки карточки.

4. Composables

Для выделения повторяющегося функционала созданы три composable-файла в папке `src/composables/`.

useApi.js - универсальное управление состоянием асинхронного запроса (loading, error, data). Используется на всех страницах каталога.

```
// src/composables/useApi.js
import { ref } from 'vue'

export function useApi(apiFn) {
  const data = ref(null)
  const loading = ref(false)
  const error = ref(null)

  async function execute(...args) {
    loading.value = true
    error.value = null
    try {
      const res = await apiFn(...args)
      data.value = res.data
    } catch (e) {
      error.value = e.message
    } finally {
      loading.value = false
    }
  }

  return { data, loading, error, execute }
}
```

useFilters.js - общая логика поиска и фильтрации по категории. Применяется в `ModelsView.vue` И `DatasetsView.vue`.

```
// src/composables/useFilters.js
import { ref, computed } from 'vue'

export function useFilters(items) {
  const search = ref("")
  const category = ref("")

  const filtered = computed(() =>
    (items.value || []).filter(item => {
      const matchSearch = !search.value ||
        item.title.toLowerCase().includes(search.value.toLowerCase())
      const matchCategory = !category.value || item.category === category.value
      return matchSearch && matchCategory
    })
  )
}
```

```

    })
  )

  return { search, category, filtered }
}

```

useLike.js - общая логика лайков с персистентностью через localStorage. Используется в компоненте ItemCard.vue и на странице AccountView.vue.

```

// src/composables/useLike.js
import { ref } from 'vue'

export function useLike(storageKey) {
  const liked = ref(
    JSON.parse(localStorage.getItem(storageKey) || '[]')
  )

  function toggle(id) {
    const idx = liked.value.indexOf(id)
    if (idx === -1) liked.value.push(id)
    else liked.value.splice(idx, 1)
    localStorage.setItem(storageKey, JSON.stringify(liked.value))
  }

  const isLiked = (id) => liked.value.includes(id)

  return { liked, toggle, isLiked }
}

```

Пример использования composables в ModelsView.vue:

```

<script setup>
import { onMounted } from 'vue'
import { useApi } from '@/composables/useApi'
import { useFilters } from '@/composables/useFilters'
import { useLike } from '@/composables/useLike'
import { getModels } from '@/api/models'
import ItemCard from '@/components/ItemCard.vue'

const { data: models, loading, execute } = useApi(getModels)
const { search, category, filtered } = useFilters(models)
const { isLiked, toggle } = useLike('likedModelIds')

onMounted(() => execute())
</script>

```

```
<template>
  <input v-model="search" placeholder="Поиск..." />
  <div v-if="loading">Загрузка...</div>
  <ItemCard
    v-for="item in filtered"
    :key="item.id"
    :item="item"
    :liked="isLiked(item.id)"
    @like="toggle"
  />
</template>
```

Вывод

В ходе лабораторной работы приложение ML Hub, разработанное в рамках ЛР1 и ЛР2 на HTML/CSS/JavaScript, успешно мигрировано на фреймворк Vue.JS3. Были реализованы все требуемые пункты задания: подключён vue-router с навигационными guards, настроена работа с внешним API через axios с базовым инстансом и интерцептором авторизации, приложение разделено на переиспользуемые компоненты (AppNavbar, ItemCard, SearchBar, FiltersBar, LoadingState) и страницы-views, а повторяющийся функционал вынесен в три composable-файла (useApi, useFilters, useLike). Миграция на Vue позволила устранить дублирование кода между страницами и обеспечить реактивное обновление интерфейса без прямой работы с DOM